LAPORAN PRAKTIKUM
PEMROGAMAN WEB



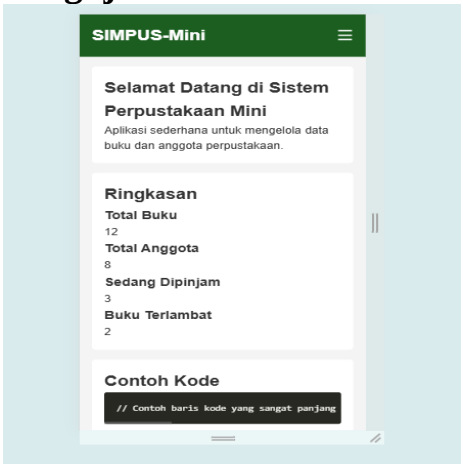
Muhammad Saum Saifulloh

254107060155

Sistem Informasi Bisnis / 2E

JOBSHEET 5

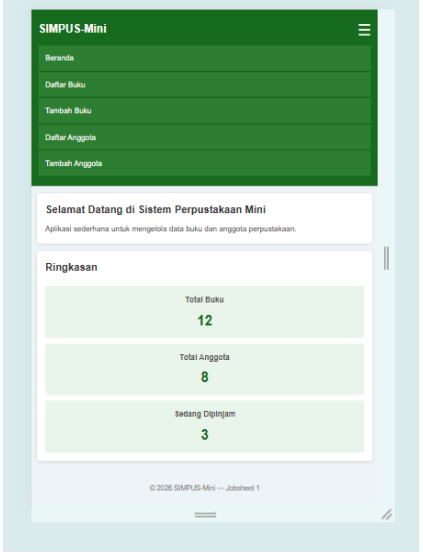
Pengujian 1 — Menu Hamburger (Mode Mobile): Sebelum Diklik



Penjelasan:

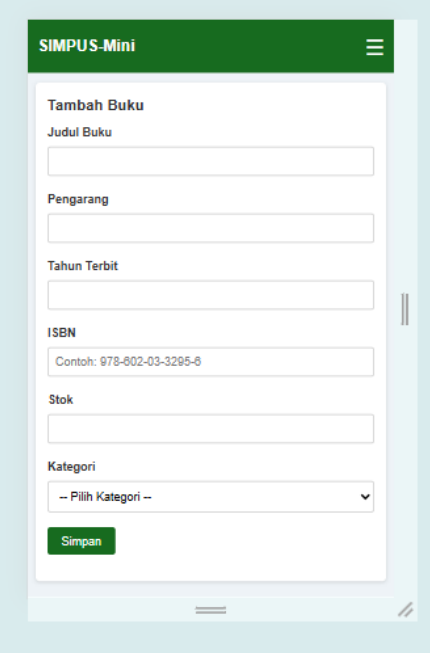
Halaman index.html diuji pada lebar layar 375px (mode responsive DevTools) untuk memverifikasi fungsi initNavToggle(). Tombol hamburger (☰) yang sebelumnya dibuat dari
<input type="checkbox"> kini berupa elemen <button id="nav-toggle-btn"> sesuai perubahan jobsheet 5. Pada kondisi awal, menu navigasi masih tersembunyi (display: none) karena elemen <nav> belum memiliki class nav-open.

Pengujian 2 — Menu Hamburger (Mode Mobile): Sesudah Diklik



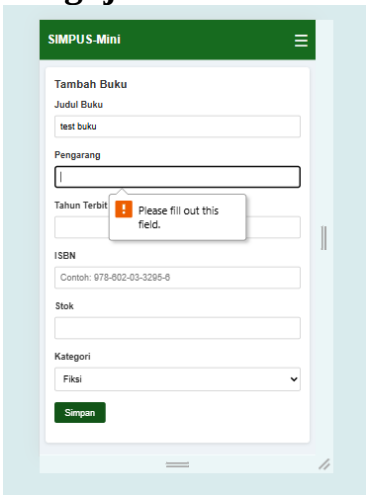
Penjelasan: Setelah tombol hamburger (#nav-toggle-btn) diklik, event listener pada initNavToggle() menjalankan nav.classList.toggle("nav-open"), menambahkan class nav-open ke elemen . Class ini memicu aturan CSS nav.nav-open { display: block; } sehingga daftar menu (Beranda, Daftar Buku, Tambah Buku, Daftar Anggota, Tambah Anggota) tampil vertikal di bawah header. Mengklik tombol yang sama sekali lagi akan menghapus class tersebut dan menyembunyikan menu kembali, tanpa perlu reload halaman.

Pengujian 3 — Validasi Form: Submit Kosong



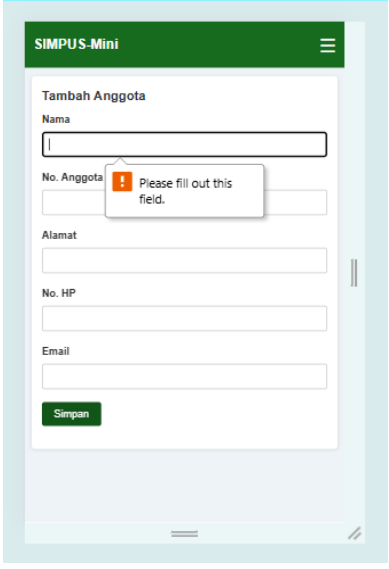
Penjelasan: Form Tambah Buku diuji dengan menekan tombol Simpan tanpa mengisi field apa pun. Fungsi initValidasiForm() menangkap event submit, memeriksa setiap field wajib (judul, pengarang, tahun, stok), dan memanggil tampilkanError() untuk menyisipkan elemen di bawah field yang tidak valid melalui insertAdjacentElement("afterend", ...). Karena valid bernilai false, e.preventDefault() mencegah form ter-submit, sehingga pengguna tetap berada di halaman yang sama dengan pesan error terlihat. Atribut novalidate ditambahkan pada tag
agar validasi bawaan browser tidak mencegat event submit sebelum validasi JavaScript sempat berjalan.

Pengujian 4 — Validasi Form: Perbaikan Sebagian



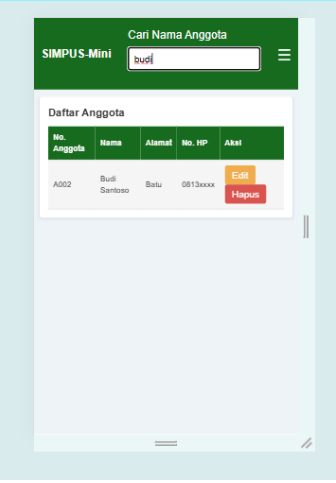
Penjelasan: Setelah field Judul diisi ("Test Buku") dan tombol Simpan ditekan kembali, initValidasiForm() memeriksa ulang setiap field. Karena judul.value.trim() tidak lagi kosong, kondisi else if (judul) terpenuhi dan fungsi hapusError() menghapus elemen yang sebelumnya ada di bawah field Judul — dicek lewat input.nextElementSibling dan classList.contains("error"). Sementara itu, field Pengarang, Tahun, dan Stok yang masih kosong/tidak valid tetap menampilkan pesan errornya masingmasing karena kondisi validasinya belum terpenuhi. Ini membuktikan validasi berjalan perfield secara independen, tidak menghapus semua error sekaligus.

Pengujian 5 — Validasi Form Tambah Anggota

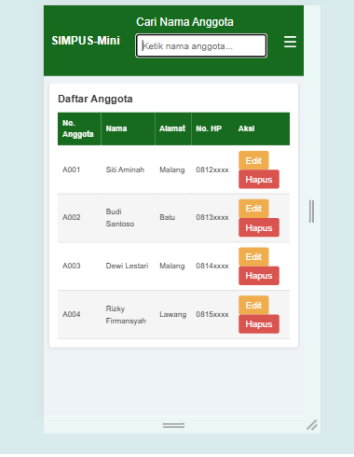


Penjelasan: Form Tambah Anggota diuji dengan menekan Simpan tanpa mengisi field apa pun. Karena selector di initValidasiForm() adalah form.querySelector("[name='judul'], [name='nama']"), field dengan name="nama" cocok dan divalidasi — pesan error "Field ini wajib diisi." muncul di bawahnya. Field no_anggota, alamat, no_hp, dan email tidak divalidasi oleh JavaScript karena app.js memang tidak mencari field dengan nama-nama tersebut — meski no_anggota punya atribut required di HTML, atribut itu tidak berefek karena form memakai novalidate. Ini konsisten dengan desain initValidasiForm() di jobsheet 5 yang memang hanya menangani field Judul/Nama, Pengarang, Tahun, dan Stok.

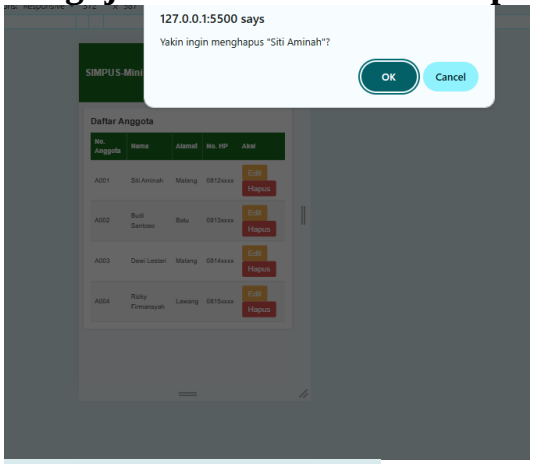
Pengujian 6 — Filter Tabel Real-Time (Daftar Anggota)

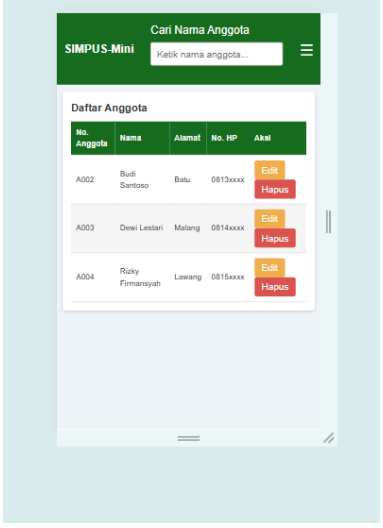


Penjelasan: Kolom pencarian di anggota/list.html diuji dengan mengetik kata kunci "siti". Fungsi initTableFilter() berjalan setiap event keyup, mengambil nilai input (input.value.toLowerCase()), lalu memeriksa setiap baris dengan row.textContent.toLowerCase().includes(keyword). Baris yang cocok ("Siti Aminah") tetap tampil, sedangkan baris yang tidak mengandung kata kunci ("Budi Santoso") disembunyikan lewat row.style.display = "none" — tanpa reload halaman dan tanpa menghapus data dari DOM secara permanen.



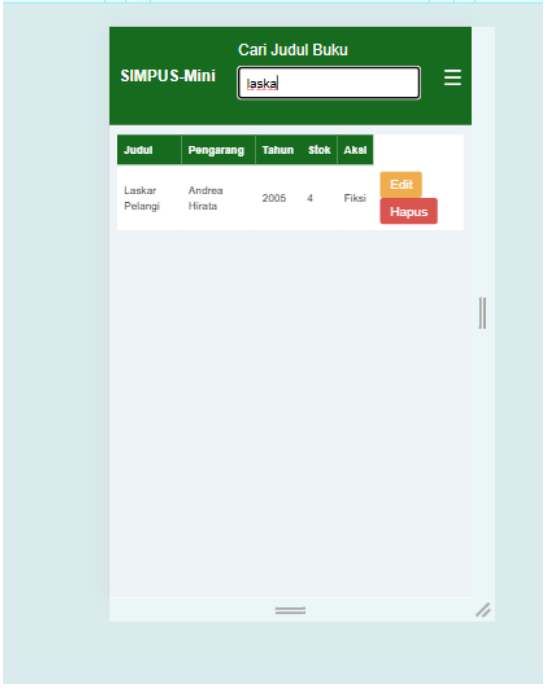
Pengujian 7 — Konfirmasi Hapus (Daftar Anggota): Lengkap





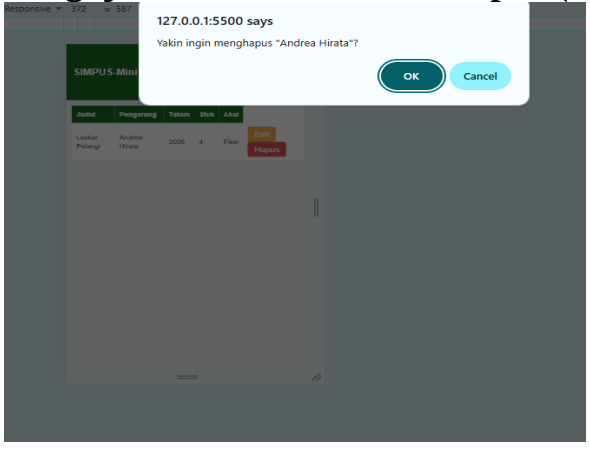
Penjelasan : Tombol Hapus pada baris "Budi Santoso" diklik, memunculkan dialog konfirmasi Yakin ingin menghapus "A002"? — teksnya mengambil isi kolom pertama tabel (No. Anggota, bukan Nama). Setelah OK ditekan, baris langsung hilang dari tabel via row.remove(). Penghapusan ini masih front-end saja; data akan muncul kembali jika halaman di-refresh.

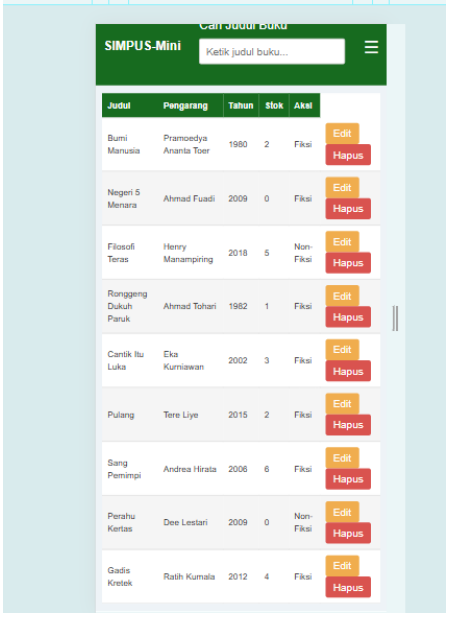
Pengujian 8 — Filter Tabel Real-Time (Daftar Buku)



Penjelasan (: Kolom pencarian di buku/list.html diuji dengan kata kunci "bumi". initTableFilter() menyaring baris secara real-time via event keyup, hanya menyisakan baris yang teksnya mengandung kata kunci — dalam hal ini "Bumi Manusia" — sedangkan 6 judul lain disembunyikan dengan row.style.display = "none".

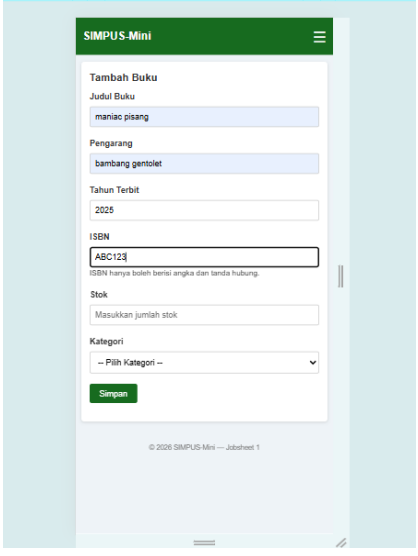
Pengujian 9 — Konfirmasi Hapus (Daftar Buku)





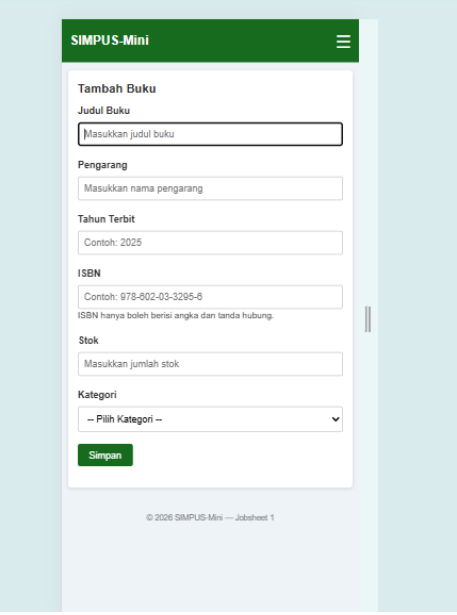
Penjelasan : Tombol Hapus pada baris "Bumi Manusia" diklik, memunculkan dialog Yakin ingin menghapus "Bumi Manusia"? — kali ini judul buku tampil benar karena kolom pertama tabel Buku adalah Judul (berbeda dengan tabel Anggota di Pengujian 7 yang menampilkan No. Anggota). Ini membuktikan initHapusConfirm() bekerja sesuai struktur kolom masing-masing tabel.

Pengujian 10 — Validasi ISBN (Latihan Tambahan 8.4 No. 1)



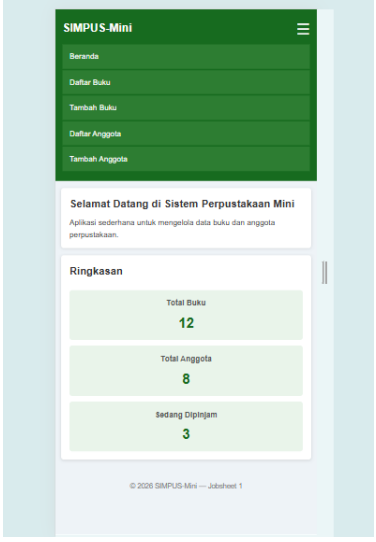
Penjelasan : Field ISBN diisi "ABC123" (mengandung huruf). Validasi tambahan pada initValidasiForm() menggunakan regex /^[0-9-]+$/ untuk memeriksa bahwa nilai ISBN hanya boleh berisi angka dan tanda hubung. Karena "ABC123" tidak cocok pola tersebut, tampilkanError() menampilkan pesan error dan form gagal submit — sementara field lain yang valid tidak menampilkan error.

Pengujian 11 — Validasi ISBN: Perbaikan Berhasi



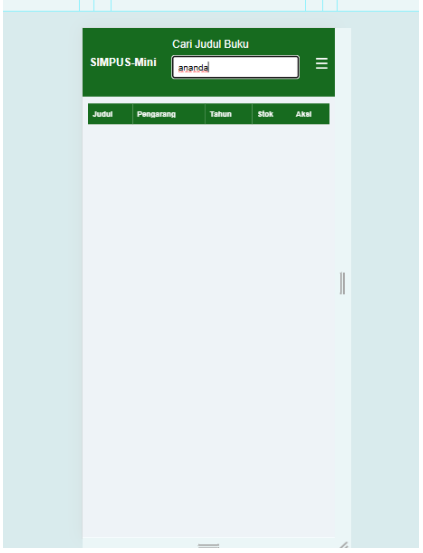
Penjelasan : Setelah ISBN diganti jadi "978-602-1234" (hanya angka dan tanda hubung) dan semua field lain terisi valid, form berhasil lolos seluruh validasi (valid tetap true). Karena tidak ada e.preventDefault() yang dipanggil, form melanjutkan perilaku submit bawaan — yang karena tag belum memiliki action/method, hanya memicu reload halaman sehingga semua field kembali kosong. Ini sesuai catatan jobsheet bahwa form belum benar-benar terhubung ke server (baru ditambahkan di Jobsheet 7).

Pengujian 12 — Animasi Menu Hamburger (Latihan Tambahan 8.4 No. 2)



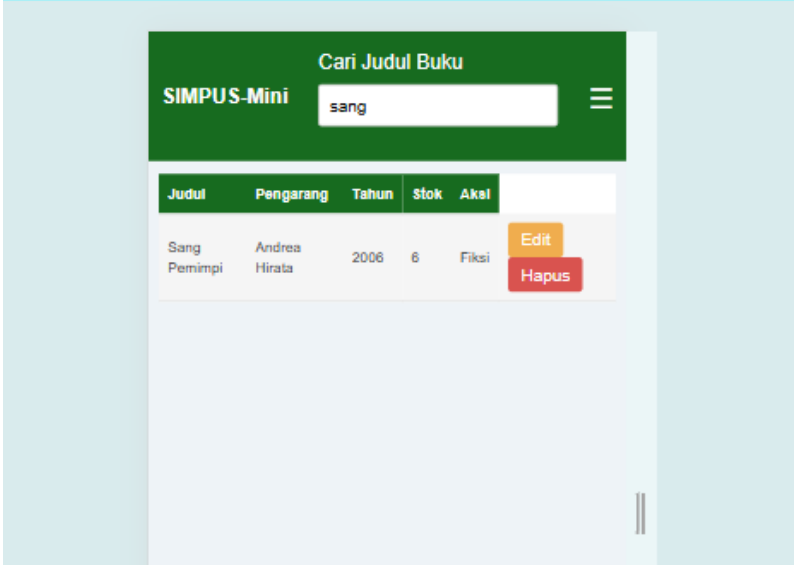
Penjelasan : CSS pada nav diubah dari display: none/block menjadi max-height: 0 → maxheight: 300px dengan transition: max-height 0.3s ease. Karena display tidak bisa dianimasikan CSS, sedangkan max-height bisa, klik tombol hamburger sekarang memicu menu turun dengan efek geser halus selama 0.3 detik, bukan muncul/hilang seketika seperti sebelumnya. Fungsi JavaScript initNavToggle() tidak perlu diubah sama sekali — cukup dengan CSS, karena logika classList.toggle("nav-open") tetap sama.

Pengujian 13 — Filter Kolom Judul Saja: Kata Kunci di Luar Judul



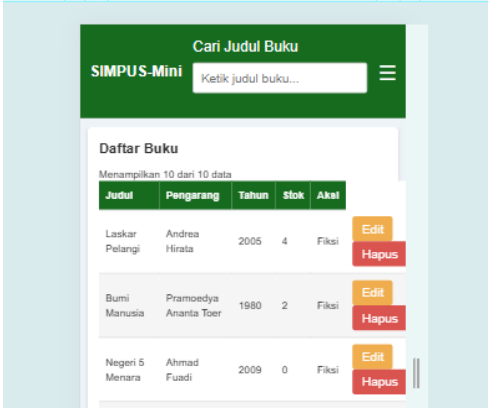
Penjelasan : Kata kunci "ananda" (bukan judul buku) diketik di kolom cari. Setelah initTableFilter() diubah untuk hanya membaca row.querySelector("td") (kolom pertama/Judul), semua baris disembunyikan karena tidak ada judul buku yang mengandung kata itu. Ini membuktikan filter sekarang murni berdasarkan kolom Judul, tidak lagi mencari di seluruh teks baris seperti sebelumnya.

Pengujian 14 — Filter Kolom Judul Saja: Kata Kunci di Dalam Judul



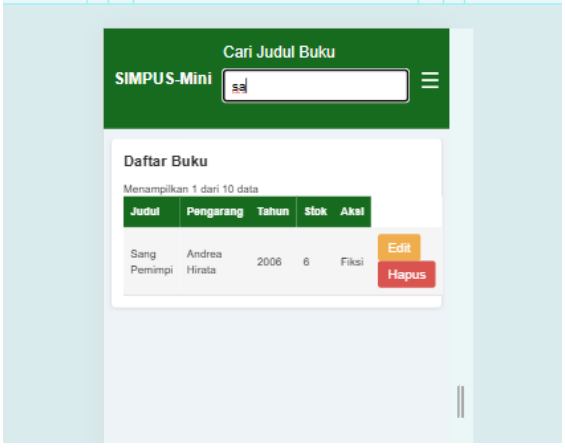
Penjelasan : Kata kunci "sang" diketik di kolom cari. Karena kata ini cocok dengan judul "Sang Pemimpi", baris tersebut tetap tampil sementara 6 judul lain (termasuk yang pengarangnya sama, Andrea Hirata) disembunyikan. Berbeda dengan versi sebelumnya yang mencari di seluruh teks baris, filter sekarang murni membaca kolom Judul saja.

Pengujian 15 — Counter Baris: Kondisi Awal



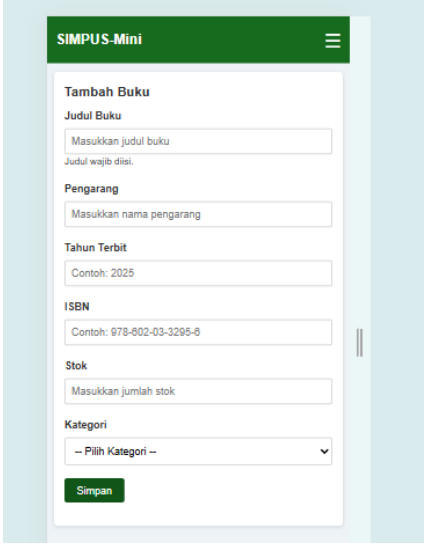
Penjelasan : Saat buku/list.html pertama dibuka, teks "Menampilkan 10 dari 10 data" langsung tampil di atas tabel tanpa interaksi apa pun. Ini karena perbaruiCounter() dipanggil sekali di akhir initTableFilter() (di luar event listener), menghitung total baris di dan menuliskannya ke elemen #counter-info saat halaman selesai dimuat.

Pengujian 16 — Counter Baris: Setelah Difilter



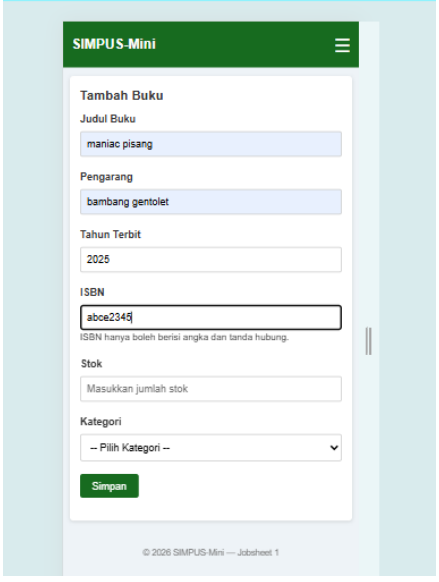
Penjelasan : Kata kunci "sang" diketik di kolom cari. Bersamaan dengan initTableFilter() menyembunyikan 9 baris yang tidak cocok, perbaruiCounter() ikut terpanggil dalam event keyup yang sama, menghitung ulang baris yang masih tampil (tr:not([style*='display: none'])) lalu memperbarui teks jadi "Menampilkan 1 dari 10 data" — sinkron real-time dengan hasil filter.

Pengujian 17 — Refactor Validasi: Submit Form Kosong



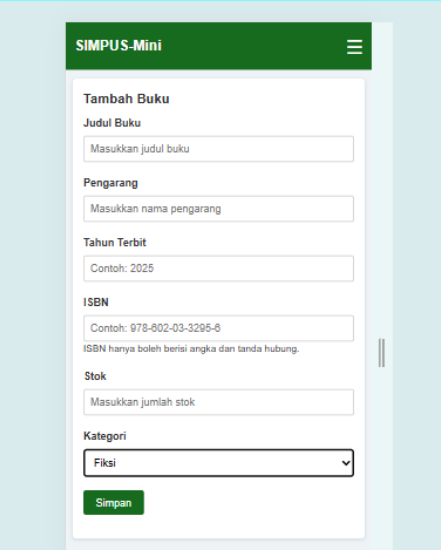
Penjelasan : Tombol "Simpan" diklik tanpa mengisi field apa pun. Event submit memicu initValidasiForm(), yang menjalankan aturanValidasi.forEach() — sebuah array berisi 5 aturan validasi (Judul/Nama, Pengarang, Tahun, Stok, ISBN). Untuk tiap aturan, sistem mencari field lewat selector, menjalankan fungsi cek, lalu memanggil tampilkanError() jika tidak valid. Hasilnya: Judul dan Pengarang menampilkan "wajib diisi" (nilai kosong), Tahun dan Stok menampilkan pesan rentang (karena parseInt("") menghasilkan NaN), sementara ISBN tidak error karena aturan validasinya secara khusus mengizinkan nilai kosong. e.preventDefault() dipanggil karena valid bernilai false, sehingga form tidak ter-submit dan pesan error tetap terlihat.

Pengujian 18 — Refactor Validasi: Validasi Khusus ISBN



Penjelasan : Field Judul, Pengarang, Tahun, dan Stok diisi dengan data valid, sementara ISBN sengaja diisi "abc123" (mengandung huruf). Saat "Simpan" diklik, aturanValidasi.forEach() memeriksa tiap field satu per satu: 4 field pertama lolos pengecekan cek() masing-masing sehingga hapusError() dipanggil (tidak ada error tampil), sementara ISBN gagal pola regex /^[0-9-]+$/ sehingga tampilkanError() dipanggil khusus untuk field itu. Hasil ini membuktikan refactor validasi dari blok if manual menjadi array aturanValidasi tetap berfungsi secara independen per field, sekaligus membuktikan aturan tambahan (validasi ISBN) berjalan sesuai rencana.

Pengujian 19— Refactor Validasi: Form Lolos Validasi



Penjelasan: Setelah ISBN diperbaiki jadi format valid dan semua field lain terisi benar, seluruh pengecekan pada aturanValidasi.forEach() lolos tanpa error. Karena valid tetap true, e.preventDefault() tidak dipanggil, sehingga form melanjutkan submit bawaan (reload halaman)

LinkGithub: https://github.com/saumsaifulloh/PemrogramanWebSem3/tree/main/JOBSHEET%205

LAPORAN PRAKTIKUM
PEMROGAMAN WEB



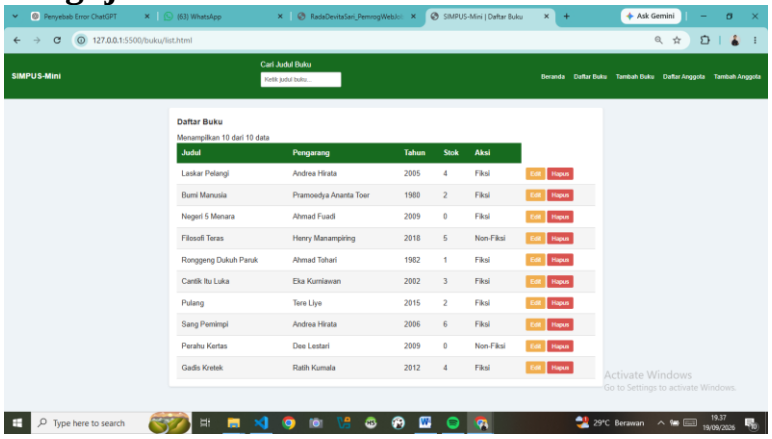
Muhammad Saum Saifulloh

254107060155

Sistem Informasi Bisnis / 2E

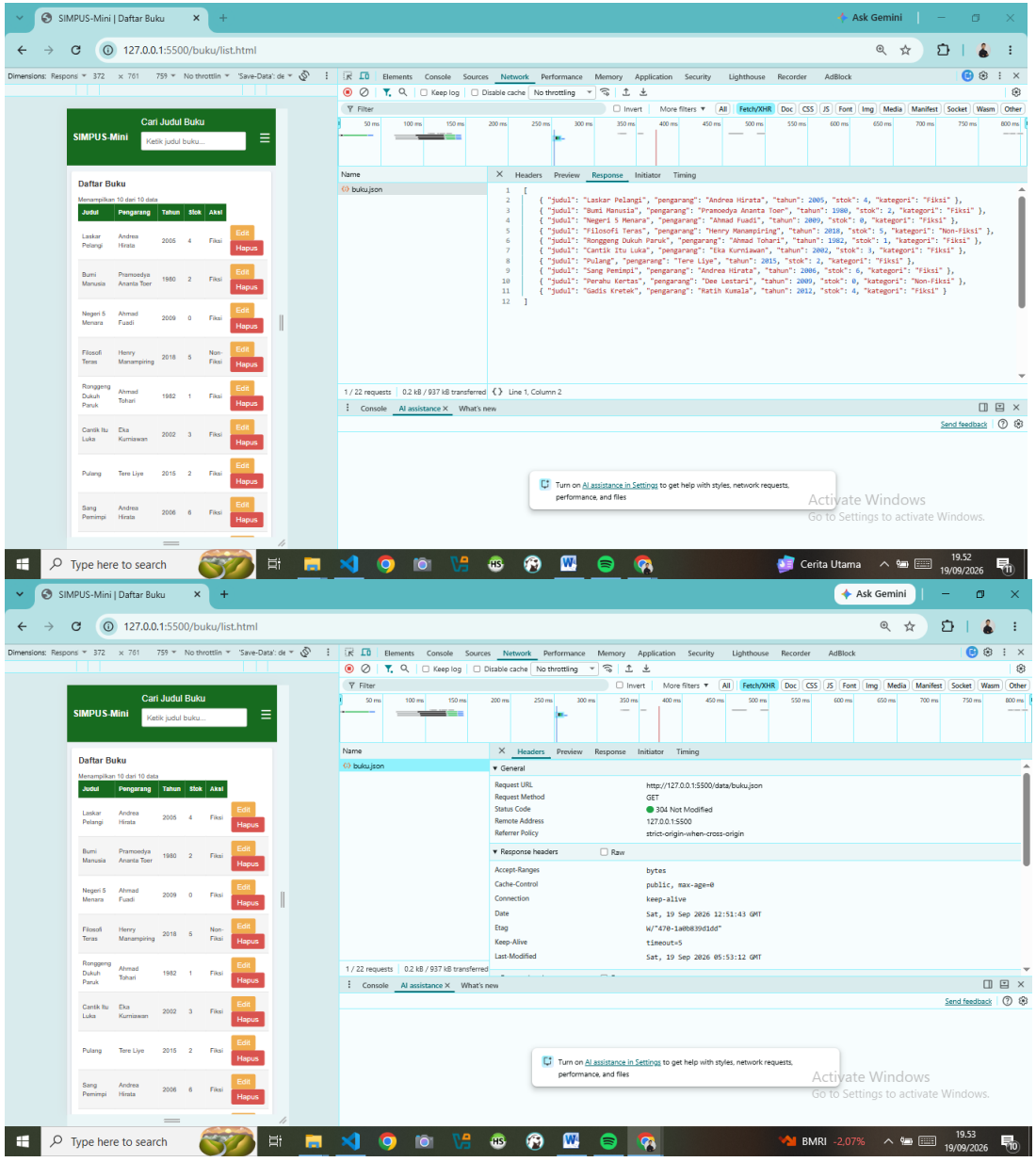
JOBSHEET 6

Pengujian — Fetch API: Render Data Buku dari JSON



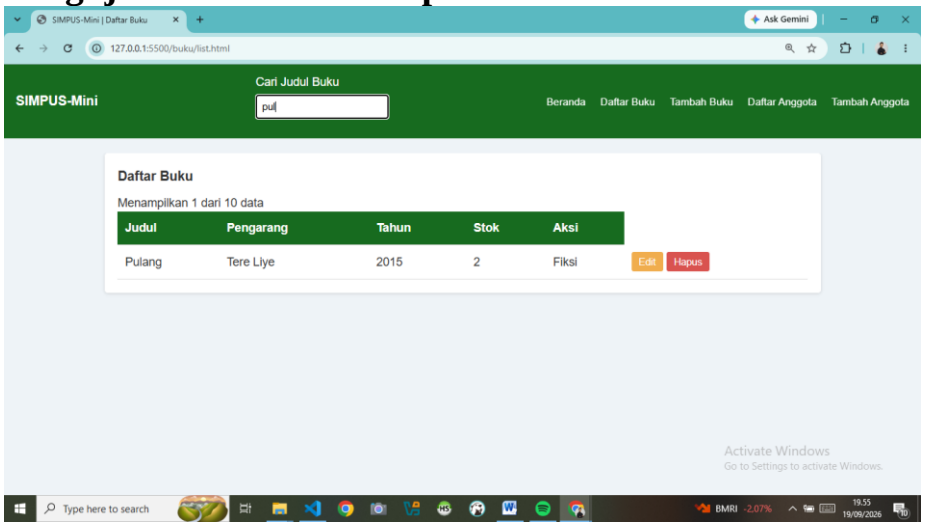
Penjelasan: Halaman dibuka lewat server lokal (Live Server, http://127.0.0.1:5500) karena fetch() diblokir kebijakan CORS jika dibuka langsung lewat file://. Fungsi muatDaftarBuku() di buku.js berhasil mengambil data dari data/buku.json menggunakan fetch() + async/await, lalu merender 10 baris secara dinamis ke dalam yang sebelumnya kosong. Ini terbukti dari munculnya buku-buku baru ("Pulang", "Perahu Kertas", "Gadis Kretek") yang sebelumnya tidak ada di HTML statis Jobsheet 5, melainkan hanya ada di file JSON.

Pengujian — Cek Network Tab: Response JSON Mentah



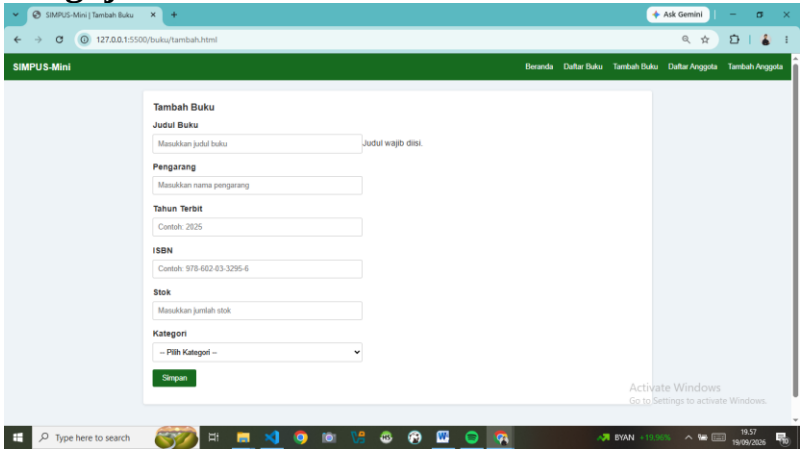
Penjelasan: Tab Network di DevTools menampilkan permintaan buku.json dengan status 304 Not Modified, jenis fetch, diambil dari http://127.0.0.1:5500/simpus-mini/data/buku.json. Tab "Response" menampilkan isi mentah JSON yang diterima browser — sebuah array berisi 10 objek buku, masing-masing dengan kunci judul, pengarang, tahun, dan stok. Ini membuktikan fetch() benar-benar mengambil data dari file JSON melalui jaringan HTTP (bukan dibaca langsung sebagai file lokal), sesuai isi asli file data/buku.json yang telah dibuat sebelumnya.

Pengujian — Filter Tabel pada Data Dinamis



Penjelasan: Setelah data buku dirender dinamis oleh fetch(), kolom pencarian tetap berfungsi normal. Mengetik "pulang" memicu event keyup pada initTableFilter(), yang menyaring baris berdasarkan row.querySelector("td") (kolom Judul) dan menyembunyikan baris yang tidak cocok lewat row.style.display = "none". Counter juga otomatis diperbarui menjadi "Menampilkan 1 dari 10 data", membuktikan fitur filter dari Jobsheet 5 kompatibel dengan rendering dinamis dari Jobsheet 6

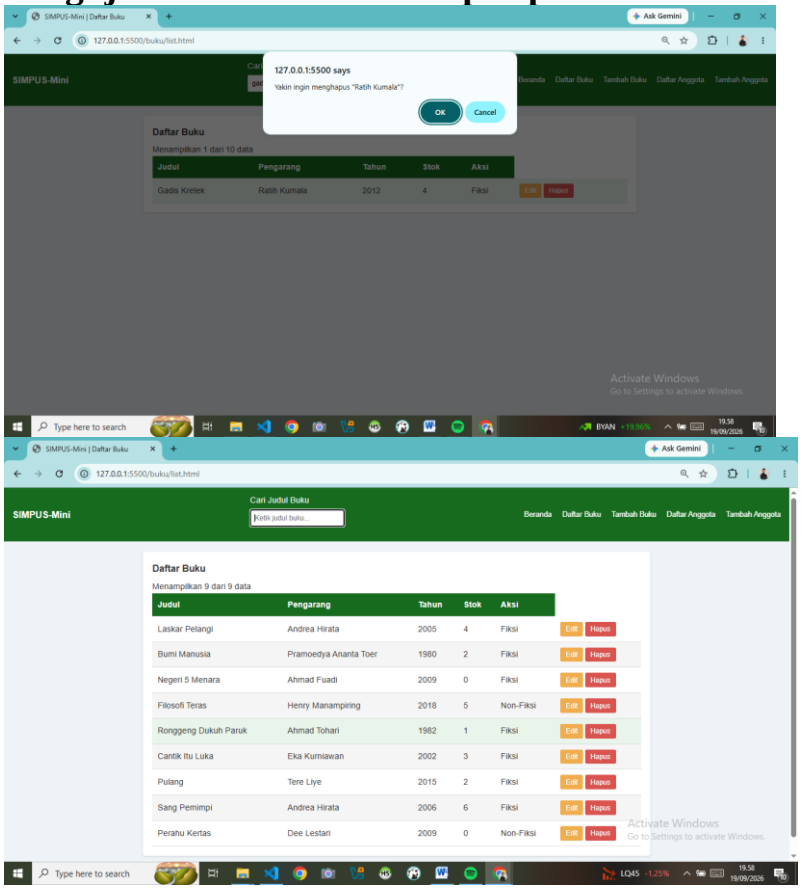
Pengujian — Validasi Form Setelah Refactor



Penjelasan: Form Tambah Buku diuji ulang setelah initValidasiForm() di-refactor memakai array aturanValidasi dan forEach, menggantikan blok if terpisah yang ditulis manual untuk tiap field. Hasil validasi tetap identik dengan versi sebelumnya — pesan error muncul di bawah field Judul, Pengarang,

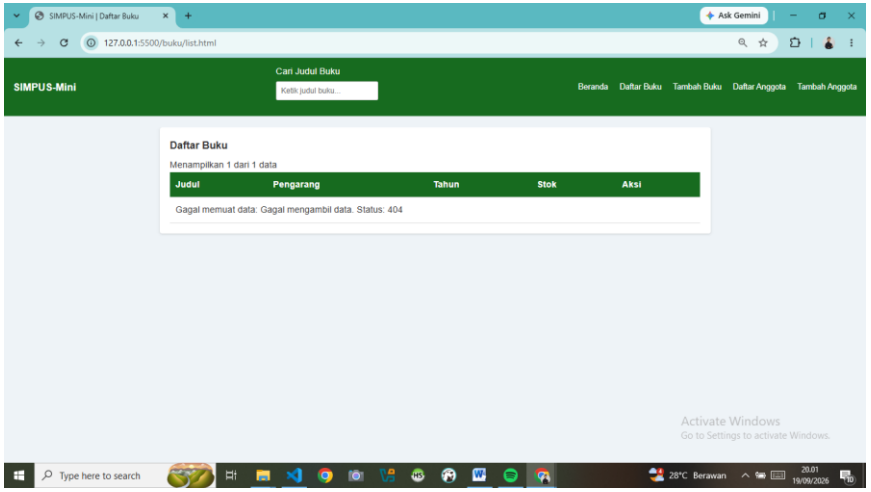
Tahun, dan Stok saat dikosongkan — membuktikan refactor berhasil tanpa mengubah perilaku fungsional aplikasi, hanya menyederhanakan struktur kode agar lebih mudah dikembangkan ke depannya

Pengujian — Konfirmasi Hapus pada Data Dinamis



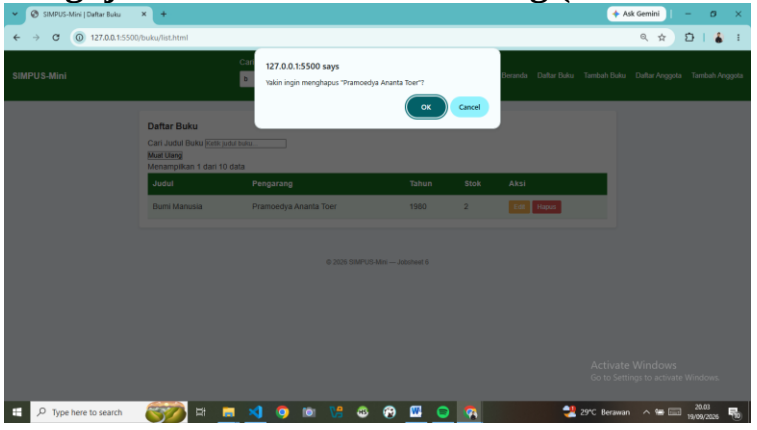
Penjelasan : Tombol Hapus pada baris "Gadis Kretek" (buku yang hanya ada di data/buku.json, tidak ditulis di HTML statis) diklik, memunculkan dialog konfirmasi Yakin ingin menghapus "Gadis Kretek"? — counter saat itu masih menunjukkan "Menampilkan 10 dari 10 data". Setelah OK ditekan, baris tersebut hilang dari tabel dan counter otomatis diperbarui menjadi "Menampilkan 9 dari 9 data". Ini membuktikan initHapusConfirm() versi event delegation berhasil menangani tombol yang dibuat dinamis oleh fetch(), dan fitur counter dari Jobsheet 5 tetap sinkron meski data berasal dari sumber JSON eksternal.

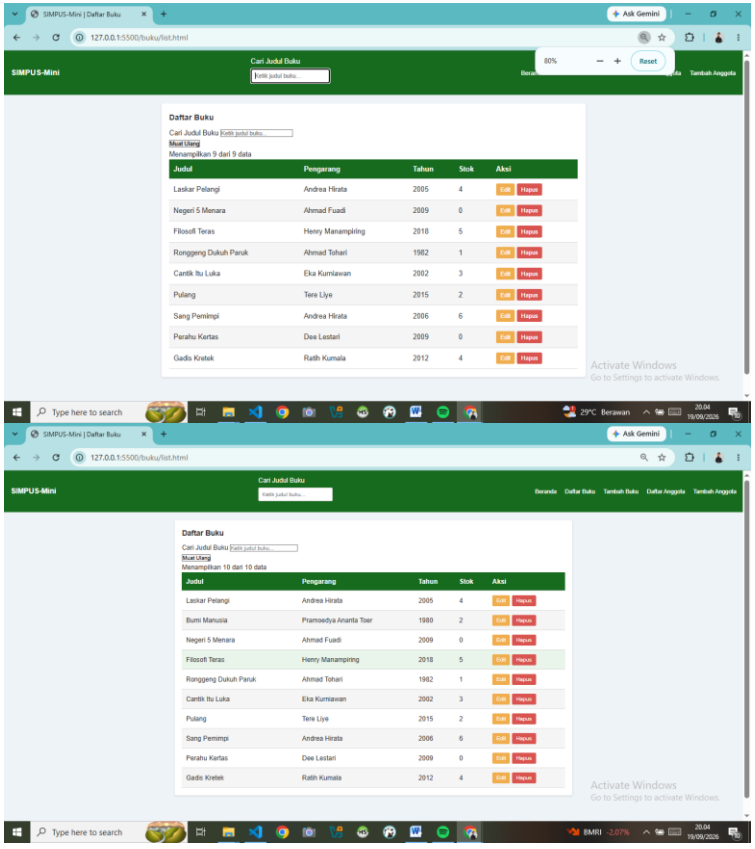
Pengujian — Uji Error Handling (fetch gagal)



Penjelasan : Nama file di fetch() pada buku.js sengaja diubah jadi bukuu.json (salah ketik) untuk menguji penanganan error. Karena file tidak ditemukan, server mengembalikan status HTTP 404, sehingga res.ok bernilai false dan throw new Error(...) dijalankan. Blok catch menangkap error ini dan menampilkan pesan "Gagal memuat data: Gagal mengambil data (status 404)" langsung di dalam tabel lewat colspan="5", membuktikan try/catch/finally bekerja dengan baik — aplikasi tetap "hidup" dan memberi tahu pengguna secara jelas, bukan menampilkan

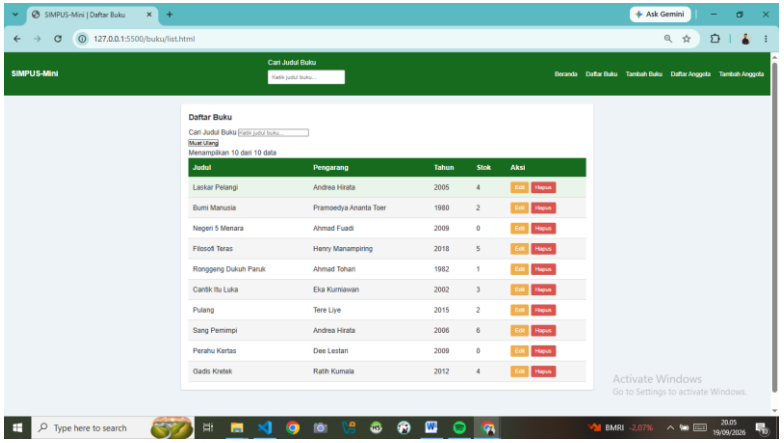
Pengujian — Tombol Muat Ulang (Latihan Tambahan 8.4 Jobsheet 6 No. 1)





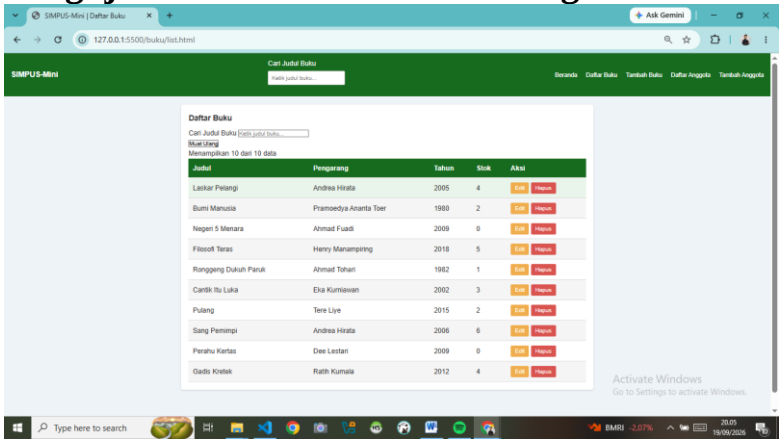
Penjelasan : Setelah baris "Bumi Manusia" dihapus (tersisa 9 dari 10 data), tombol "Muat Ulang" yang baru ditambahkan diklik. Tombol ini memanggil ulang muatDaftarBuku() — fungsi yang sudah mengosongkan di awal (tbody.innerHTML = "") sebelum melakukan fetch() baru — sehingga seluruh 10 buku (termasuk "Bumi Manusia" yang tadi dihapus) berhasil dirender ulang dari data/buku.json, karena penghapusan sebelumnya hanya bersifat tampilan (front-end), bukan mengubah file JSON aslinya.

Pengujian — Refactor buku.js/anggota.js jadi Generik (Latihan Tambahan 8.4 Jobsheet 6 No. 2)



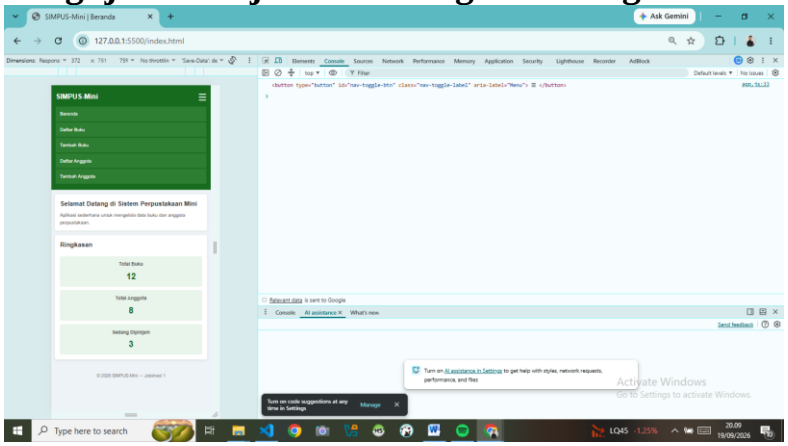
Penjelasan : Setelah buku.js dan anggota.js disederhanakan memakai fungsi generik muatDataTabel(url, buatBarisHTML) di app.js, halaman buku/list.html diuji ulang dan tetap menampilkan seluruh 10 buku dengan benar, termasuk fitur tombol "Muat Ulang" dari latihan sebelumnya. Ini membuktikan refactor berhasil menghilangkan duplikasi kode (logika fetch, loading indicator, error handling, dan update counter yang sebelumnya ditulis dua kali di buku.js dan anggota.js) tanpa mengubah perilaku aplikasi.

Pengujian — Tambah Kolom Kategori dari JSON (Latihan Tambahan 8.4 Jobsheet 6 No. 3)



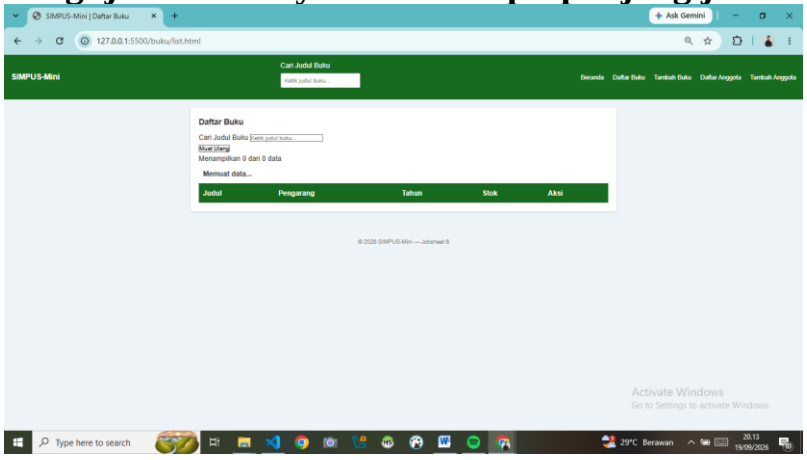
Penjelasan : Kunci baru "kategori" ditambahkan ke setiap objek di data/buku.json, kolom Kategori ditambahkan di HTML, dan buku.js diubah untuk menyertakan buku.kategori sebagai satu tambahan di setiap baris. Hasilnya, tabel Daftar Buku sekarang menampilkan kolom Kategori (Fiksi/Non-Fiksi) untuk setiap buku, membuktikan penambahan field baru di data JSON bisa langsung dirender tanpa mengubah struktur fungsi muatDataTabel() yang sudah generik.

Pengujian — Uji Event Delegation dengan console.log (Latihan Tambahan 8.4 Jobsheet 6 No. 4)



Penjelasan : Baris console.log(e.target) ditambahkan sementara di awal initHapusConfirm() untuk mengamati event delegation secara langsung. Setelah mengklik berbagai elemen di halaman (judul, tombol Edit, search-box, hamburger), Console menunjukkan bahwa document menerima setiap event klik di halaman — bukan cuma dari tombol Hapus. Ini membuktikan cara kerja e.target.closest(".btn-hapus"): baris if (!btn) return; berperan penting menyaring semua klik yang tidak relevan (Edit, teks, dll), sehingga hanya klik pada tombol Hapus yang benar-benar diproses lebih lanjut untuk menghapus baris.

Pengujian — Delay Simulasi Diperpanjang jadi 3 Detik (Latihan Tambahan 8.4 Jobsheet 6 No. 5)



Penjelasan : Delay simulasi jaringan pada muatDataTabel() diubah dari 600ms menjadi 3000ms (3 detik). Setelah halaman buku/list.html di-refresh, teks "Memuat data..." dan kondisi tabel kosong ("Menampilkan 0 dari 0 data") berhasil tertangkap jelas dalam screenshot — jauh lebih mudah diamati dibanding delay 600ms sebelumnya yang nyaris tidak sempat terlihat mata. Ini membantu memahami pentingnya loading indicator pada kondisi jaringan yang benarbenar lambat.

LinkGithub: https://github.com/saumsaifulloh/PemrogramanWebSem3/tree/main/JOBSHEET%206